C LANGUAGE

C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph), located in the U.S.A.

Dennis Ritchie is known as the founder of the c language.

It was developed to overcome the problems of previous languages such as B, BCPL, etc.

Initially, C language was developed to be used in UNIX operating system. It inherits many features of previous languages such as B and BCPL.

Features of C Language

C is the widely used language. It provides many features that are given below.

1. Simple
2. Machine Independent or Portable
3. Mid-level programming language
4. structured programming language
5. Rich Library
6. Memory Management
7. Fast Speed
8. Pointers
9. Recursion
10. Extensible

1) Simple

C is a simple language in the sense that it provides a structured approach (to break the problem into parts), the rich set of library functions, data types, etc.

2) Machine Independent or Portable

Unlike assembly language, c programs can be executed on different machines with some machine specific changes. Therefore, C is a machine independent language.

3) Mid-level programming language

Although, C is intended to do low-level programming. It is used to develop system applications such as kernel, driver, etc. It also supports the features of a high-level language. That is why it is known as mid-level language.

4) Structured programming language

C is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify. Functions also provide code reusability.

5) Rich Library

C provides a lot of inbuilt functions that make the development fast.

6) Memory Management

It supports the feature of dynamic memory allocation. In C language, we can free the allocated memory at any time by calling the free() function.

7) Speed

The compilation and execution time of C language is fast since there are lesser inbuilt functions and hence the lesser overhead.

8) Pointer

C provides the feature of pointers. We can directly interact with the memory by using the pointers. We can use pointers for memory, structures, functions, array, etc.

9) Recursion

In C, we can call the function within the function. It provides code reusability for every function. Recursion enables us to use the approach of backtracking.

10) Extensible

C language is extensible because it can easily adopt new features.

First C Program

General Overview of a Simple C Program's Structure:

The general architecture of a simple C program typically consists of several vital components. Below is an outline of the essential elements and their purposes:

- Header Files:

  The #include directives at the beginning of the program are used to include header files. Header files provide function prototypes and definitions that allow the C compiler to understand the functions used in the program.

- Main Function:

  Every C program starts with the main function. It is the program's entry point, and execution starts from here. The main function has a return type of int, indicating that it should return an integer value to the operating system upon completion.

- Variable Declarations:

  Before using any variables, you should declare them with their data types. This section is typically placed after the main function's curly opening brace.

- Statements and Expressions:

  This section contains the actual instructions and logic of the program. C programs are composed of statements that perform actions and expressions that compute values.

- Comments:

  Comments are used to provide human-readable explanations within the code. They are not executed and do not affect the program's functionality. In C, comments are denoted by // for single-line comments and /* */ for multi-line comments.

- Functions:

  C programs can include user-defined functions and blocks of code that perform specific tasks. Functions help modularize the code and make it more organized and manageable.

- Return Statement:

  Use the return statement to terminate a function and return a value to the caller function. A return statement with a value of 0 typically indicates a successful execution in the main function, whereas a non-zero value indicates an error or unexpected termination.

- Standard Input/Output:

  C has library functions for reading user input (scanf) and printing output to the console (printf). These functions are found in C programs and are part of the standard I/O library (stdio.h header file). It is essential to include these fundamental features correctly while writing a simple C program to ensure optimal functionality and readability.

Basic Program of C

```
#include<stdio.h>---------- Standard Input Output.
#include<conio.h>---------- Console Input Output.
void main() {---------- No return type.
clrscr();---------- Clear Screen
printf("Hello World");---------- Print
getch();---------- Output Screen Holding
}
```

- #:- Preprocessor directive (Statement start with # called preprocessing statement).
- Include:- Keyword.
- Stdio:- Standard Input Output.
- Conio:- Console Input Output.
- .h:- Extension of a header file.
- Void:- Data Type.
- Main:- Function Name.
- {}:- The body of the main function represents a particular block.
- ;:- Terminator. <>:- Diamond Braces (Angular) finds the location of a file.
- Stdio.h:- Standard input output header file which consists of the declaration of predefine functions like printf() & scanf().
- Conio.h:- Console input output header file which consists of the declaration of predefine functions like clrscr() & getch().
- Main:- main() is a function from which the execution of a program is started.
- .c:- is a source file.
- clrscr():- Clear Screen.
- getch():- Screen Hold.

printf() and scanf() in C

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

printf() function

The printf() function is used for output. It prints the given statement to the console.

The syntax of printf() function is given below:

```
printf("format string",argument_list);
```

scanf() function

The scanf() function is used for input. It reads the input data from the console.

```
scanf("format string",argument_list);
```

Variables in C

Variables are key building elements of the C programming language used to store and modify data in computer programs. A variable is a designated memory region that stores a specified data type value. Each variable has a unique identifier, its name, and a data type describing the type of data it may hold.

Syntax:

The syntax for defining a variable in C is as follows:

```
data_type variable_name;
```

- data_type: It represents the type of data the variable can hold. Examples of data types in C include int (integer), float (a floating-point number), char (character), double (a double-precision floating-point number),
- variable_name: It is the identifier for the variable, i.e., the name you give to the variable to access its value later in the program. The variable name must follow specific rules, like starting with a letter or underscore and consisting of letters, digits, and underscores.

  For example, to declare the integer variable age:

  int age;

- **Rules for defining variables**

In C, Variable names must follow a few rules to be valid. The following are the rules for naming variables in C:

- Allowed Characters:

  Variable names include letters (uppercase and lowercase), digits, and underscores. They must start with a letter (uppercase or lowercase) or an underscore.

- Case Sensitivity:

  C is a case-sensitive programming language. It means that uppercase and lowercase letters are considered distinct.

  For example, myVar, MyVar, and myvar are all considered different variable names.

- Keywords:

  Variable names cannot be the same as C keywords (reserved words), as they have special meanings in the language.

  For example, you cannot use int, float, char, for, while, etc., as variable names.

- Length Limitation:

  There is no standard limit for the length of variable names in C, but it's best to keep them reasonably short and descriptive.

- Spaces and Special Characters:

  Variable names cannot contain spaces or special characters (such as !, @, #, $, %, ^, &, *, (,), -, +, =, [,], {, }, |, \, /, <, >,., ?;, ', or ").

  Underscores are the only allowed special characters in variable names.

- Reserved Identifiers:

  While not strictly a rule, it is advisable to avoid specific patterns or identifiers common in libraries or standard usage.

Data Types in C

A data type specifies the type of data that a variable can store such as integer, floating, character, etc.

Data Type	Size	Description	Example
int	2 or 4 bytes	Stores whole numbers, without decimals	1
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits	1.99
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits	1.99
char	1 byte	Stores a single character/letter/number, or ASCII values	'A'

- int

```
#include <stdio.h>
int main() {
  int myNum = 1000;
  printf("%d", myNum);
  return 0;
```

- float

```
#include <stdio.h>
int main() {
  float myNum = 5.75;
  printf("%f", myNum);
  return 0;
}
```

- double

```
#include <stdio.h>
int main() {
  double myNum = 19.99;
  printf("%lf", myNum);
  return 0;
}
```

- char

```
#include <stdio.h>
int main() {
  char myGrade = 'A';
  printf("%c", myGrade);
  return 0;
}
```

Format specifier

There are different format specifiers for each data type

%d:- int

%c:-char

%f:-float

%lf:-double

%s:-string

Task use escape sequences

\n:- new line	\r:- carriage return	\a:- produce sound	\\:- print\
\t:- tab space	\b:- backspace	\":- print"	

Keywords

These are the reserved or predefined words whose meaning is already known to the compiler.

There are 32 keywords available in C-Language for 16-bit architecture as per ASVSI standard.

1)Auto	2)for	3)switch
4)break	5)goto	6)typedef
7)case	8)if	9)union
10)char	11)int	12)unsigned
13)const	14)long	15)void
16)continue	17)register	18)volatile
19)default	20)return	21)while
22)do	23)short	24)double
25)singed	26)static	27)else
28)size of	29)enum	30)struct
31)extern	32)float	

C Operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operator
- Bitwise Operators
- Ternary or Conditional Operators

- **Arithmetic Operators:**

Arithmetic operators carry out fundamental mathematical operations. The arithmetic operators in C are as follows:

Addition Operator (+): The addition operator adds two operands together.

Syntax:

It has the following syntax:

```
result = operand1 + operand2;
```

Example:

```
int a = 5;

int b = 3;

int result = a + b;
```

Subtraction Operator (-): The second operand is subtracted from the first operand via the subtraction operator.

Syntax:

It has the following syntax:

```
result = operand1 - operand2;
```

Example:

```
int a = 8;

int b = 3;

int result = a - b;
```

Multiplication Operator (*): This operator is used to multiply the two operands.

Syntax:

It has the following syntax:

```
result = operand1 * operand2;
```

Example:

```
int a = 4;

int b = 5;

int result = a * b;
```

Output:

result = 20

Division Operator (/): The first operand and the second operand are divided using the division operator.

Syntax:

It has the following syntax:

```
result = operand1 / operand2;
```

Example:

```
int a = 10;

int b = 2;

int result = a / b;
```

Modulus Operator (%): The modulus operator determines the remainder of the division between two operands.

Syntax:

It has the following syntax:

result = operand1 % operand2;

Example:

```
int a = 10;

int b = 3;

int result = a % b;
```

Output:

result = 1

- **Relational Operators:**

Relational operators assess the relationship between values by comparing them. They return either true (1) or false (0). The relational operators in C are as follows:

Equality Operator (==): If two operands are equal, the equality operator verifies this.

Syntax:

It has the following syntax:

```
result = operand1 == operand2;
```

Example:

```
int a = 5;

int b = 5;

int result = a == b;
```

Output:

result=1 (true)

Inequality Operator (!=): The inequality operator determines whether two operands are equal or not.

Syntax:

It has the following syntax:

```
result = operand1 != operand2;
```

Example:

```
int a = 5;

int b = 3;

int result = a != b;
```

Output:

result=1 (true)

Greater than Operator (>): The greater than operator determines if the first operand exceeds the second operand.

Syntax:

It has the following syntax:

```
result = operand1 > operand2;
```

Example:

```
int a = 7;

int b = 4;

int result = a > b;
```

Output:

result=1 (true)

Less than Operator (<): The less-than operator determines if the first operand less is than the second operand.

Syntax:

It has the following syntax:

```
result = operand1 < operand2;
```

Example:

```
int a = 2;

int b = 6;

int result = a < b;
```

Output:

result=1 (true)

Greater than or Equal to Operator (>=): The greater than or equal to operator determines if the first operand is more than or equal to the second operand.

Syntax:

It has the following syntax:

```
result = operand1 >= operand2;
```

Example:

```
int a = 5;

int b = 5;

int result = a >= b;
```

Output:

result=1 (true)

Less than or Equal To Operator (<=): The less than or equal to operator determines if the first operand must be less than or equal to the second operand.

Syntax:

It has the following syntax:

result = operand1 <= operand2;

Example:

int a = 3;

int b = 6;

int result = a <= b;

Output:

result=1 (true)

- **Logical Operators:**

  Logical operators perform logical operations on boolean values and return either true (1) or false (0). Here are the logical operators in C:

  Logical AND Operator (&&): The logical AND operator returns true if both operands are true.

  Syntax:

  It has the following syntax:

  result = operand1 && operand2;

  Example:

  int a = 5;

  int b = 3;

  int result = (a > 3) && (b < 5);

  Output:

  result = 1 (true)

Logical OR Operator (||): The logical OR operator returns true if at least one of the operands is true.

Syntax:

It has the following syntax:

result = operand1 || operand2;

Example:

int a = 5;

int b = 3;

int result = (a > 3) || (b > 5);

Output:

result = 1 (true)

Logical NOT Operator (!): The logical NOT operator negates the value of the operand.

Syntax:

It has the following syntax:

result = !operand;

Example:

int a = 5;

int result = !(a > 3);

Output:

result = 0 (false)

- **Assignment Operator:**

  Assignment operators are used to assign values to variables. Here is some of the assignment operator in C:

  Simple Assignment Operator (=): The simple assignment operator assigns the value from the right side operands to the left side operands.

  Syntax:

  It has the following syntax:

  variable = value;

  Example:

  int a;

  a = 5;

  Output:

  No output. The value 5 is assigned to variable 'a'.

  Miscellaneous Operator:

  The sizeof operator and the comma operator fall under the miscellaneous operator category.

  sizeof Operator: The sizeof operator returns the size, in bytes, of a variable or a data type.

  Syntax:

  It has the following syntax:

  result = sizeof(variable / data type);

  Example:

  int a;

  int size = sizeof(a);

  Output:

  size = 4 // Assuming int occupies 4 bytes

Comma Operator (,): The comma operator evaluates multiple expressions and returns the value of the last expression.

Syntax:

It has the following syntax:

result = (expression1, expression2,..., expressionN);

Example:

int a = 5, b = 3;

int result = (a += 2, b *= 2, a + b);

Output:

result = 15 // a = 7, b = 6, a + b = 13

- **Bitwise Operators:**

  Bitwise operators perform operations on individual bits of the operands. Here are the bitwise operators in C:

  Bitwise AND Operator (&): The bitwise AND operator performs a bitwise AND operation on the corresponding bits of the operands.

  Syntax:

  It has the following syntax:

  result = operand1 & operand2;

  Example:

  unsigned int a = 5;

  unsigned int b = 3;

  int result = a & b;

  Output:

  result = 1

Bitwise OR Operator (|): The bitwise OR operator performs a bitwise OR operation on the corresponding bits of the operands.

Syntax:

It has the following syntax:

result = operand1 | operand2;

Example:

int a = 5;

int b = 3;

int result = a | b;

Output:

result = 7

Bitwise XOR Operator (^): The bitwise XOR operator performs a bitwise exclusive OR operation on the corresponding bits of the operands.

Syntax:

It has the following syntax:

result = operand1 ^ operand2;

Example:

unsigned int a = 5;

unsigned int b = 3;

int result = a ^ b;

Output:

result = 6

Bitwise NOT Operator (~): The bitwise NOT operator flips each bit of the operand.

Syntax:

It has the following syntax:

result = ~operand;

Example:

unsigned int a = 5;

int result = ~a;

Output:

result = -6

- **Ternary or Conditional Operator:**

  The ternary or conditional operator allows you to assign a value based on a condition.

  Syntax:

  It has the following syntax:

  result = condition ? value1 : value2;

  Example:

  int a = 5;

  int b = 3;

  int result = (a > b) ? a : b;

  Output:

  result = 5

Control Statements

- **The if Statement**

  Use the if statement to specify a block of code to be executed if a condition is true.

  Syntax
  if (condition){
  //code to be executed
  }

  Example:
  #include <stdio.h>
  int main() {
    if (20 > 18) {
      printf("20 is greater than 18");
    }
    return 0;
  }

  Output:

  20 is greater than 18

  Nested if Statement

  #include <stdio.h>
  int main()
  {
  int a, b, c;
       printf("Enter three numbers?");
      scanf("%d %d %d",&a,&b,&c);
      if(a>b && a>c)
      {
          printf("%d is largest",a);
      }
      if(b>a && b > c)
      {
          printf("%d is largest",b);
      }
  if(c>a && c>b)

```
        {
           printf("%d is largest",c);
            }
            if(a == b && a == c)
            {
                printf("All are equal");
            }
        }
```

Output

Enter three numbers?

12 23 34

34 is largest

The If Else

Use the else statement to specify a block of code to be executed if the condition is false.

```
#include <stdio.h>
int main() {
  int time = 20;
  if (time < 18) {
    printf("Good day.");
  } else {
    printf("Good evening.");
  }
  return 0;
}
```

Output

Good evening.

Nested if else statement in C

One of the fundamental constructs in programming is conditional statements. They allow a program to take different paths based on the values of certain conditions. In C, conditional statements are implemented using if-else statements.

Example:

```
#include <stdio.h>

int main() {

   int num;

printf("Enter a number: ");

scanf("%d", &num);

   if (num > 0) {

printf("%d is positive.\n", num);

       } else {

      if (num < 0) {

printf("%d is negative.\n", num);

      } else {

printf("%d is zero.\n", num);

      }

   }

   return 0;

}
```

Output:

Enter a number: 10

10 is positive.

The If else if Statement

Use the else if statement to specify a new condition if the first condition is false.

Example:

```
#include <stdio.h>

int main() {

  int time = 22;

  if (time < 10) {

    printf("Good morning.");

  } else if (time < 20) {

    printf("Good day.");

  } else {

    printf("Good evening.");

  }

  return 0;

}
```

Output:

Good evening.

If else-if ladder Statement

The if-else-if ladder statement is an extension to the if-else statement. It is used in the scenario where there are multiple cases to be performed for different conditions.

Example:

```
#include<stdio.h>

int main(){

int number=0;

printf("enter a number:");
```

```
scanf("%d",&number);

if(number==10){

printf("number is equals to 10");

}

else if(number==50){

printf("number is equal to 50");

}

else if(number==100){

printf("number is equal to 100");

}

else{

printf("number is not equal to 10, 50 or 100");

}

return 0;

}
```

Output:

```
enter a number:4

number is not equal to 10, 50 or 100
```

Switch Statement

The switch statement in C is an alternate to if-else-if ladder statement which allows us to execute multiple operations for the different possibles values of a single variable called switch variable. Here, We can define various statements in the multiple cases for the different values of a single variable.

syntax of switch statement

```
switch(expression){

case value1:
```

```
 //code to be executed;

 break;  //optional

case value2:

 //code to be executed;

 break;  //optional

.....

default:

code to be executed if all cases are not matched;

}
```

Rules for switch statement in C language

1. The switch expression must be of an integer or character type.
2. The case value must be an integer or character constant.
3. The case value can be used only inside the switch statement.
4. The break statement in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as fall through the state of C switch statement.

Example of a switch statement

```
include<stdio.h>
int main(){
int number=0;
printf("enter a number:");
scanf("%d",&number);
switch(number){
case 10:
printf("number is equals to 10");
break;
case 50:
printf("number is equal to 50");
break;
case 100:
printf("number is equal to 100");
break;
default:
printf("number is not equal to 10, 50 or 100");
}

return 0;  }
```

Output

enter a number:4

number is not equal to 10, 50 or 100

enter a number:50

number is equal to 50

Break and Default keyword in Switch statement

Let us explain and define the "break" and "default" keywords in the context of the switch statement, along with example code and output.

1. Break Keyword:

The *"break" keyword* is used within the code block of each case to terminate the switch statement prematurely. When the program encounters a *"break" statement* inside a case block, it immediately exits the *switch statement*, preventing the execution of subsequent case blocks. The *"break" statement* is crucial for avoiding switch statements' *"fall-through"* behavior.

Example:

Let's take a program to understand the use of the break keyword in C.

```
#include <stdio.h>

int main() {

int num = 3;

switch (num) {

case 1:

printf("Value is 1\n");

break; // Exit the switch statement after executing this case block

case 2:

printf("Value is 2\n");

break; // Exit the switch statement after executing this case block

case 3:
```

```
printf("Value is 3\n");
break; // Exit the switch statement after executing this case block
default:
printf("Value is not 1, 2, or 3\n");
break; // Exit the switch statement after executing the default case block
}
return 0;
}
```

Output

Value is 3

Default Keyword:

When none of the case constants match the evaluated expression, it operates as a catch-all case. If no matching case exists and a "default" case exists, the code block associated with the "default" case is run. It is often used to handle circumstances where none of the stated situations apply to the provided input.

Example:

Let's take a program to understand the use of the default keyword in C.

```
#include <stdio.h>
int main() {
int num = 5;
switch (num) {
case 1:
printf("Value is 1\n");
break;
case 2:
printf("Value is 2\n");
```

```
break;
case 3:
printf("Value is 3\n");
break;
default:
printf("Value is not 1, 2, or 3\n");
break; // Exit the switch statement after executing the default case block
}
return 0;
}
```

Output

```
Value is not 1, 2, or 3
```

Nested switch case statement

We can use as many switch statement as we want inside a switch statement. Such type of statements is called nested switch case statements. Consider the following example.

```
#include <stdio.h>
int main () {
int i = 10;
int j = 20;
 switch(i) {
case 10:
printf("the value of i evaluated in outer switch: %d\n",i);
case 20:
    switch(j) {
case 20:
```

```
            printf("The value of j evaluated in nested switch: %d\n",j);
        }
            }
            printf("Exact value of i is : %d\n", i );
            printf("Exact value of j is : %d\n", j );
            return 0;
    }
```

Output

the value of i evaluated in outer switch: 10

The value of j evaluated in nested switch: 20

Exact value of i is : 10

Exact value of j is : 20

Loops

The looping simplifies the complex problems into the easy ones. It enables us to alter the flow of the program so that instead of writing the same code again and again, we can repeat the same code for a finite number of times.

Types of C Loops

There are three types of loops in C language that is given below:

1. do while
2. while
3. for

do while loop in C

A "do-while" loop is a form of a loop in C that executes the code block first, followed by the condition. If the condition is true, the loop continues to run; else, it stops. However, whether the condition is originally true, it ensures that the code block is performed at least once.

do while loop syntax

The syntax of the C language do-while loop is given below:

```
do{
//code to be executed
}while(condition);
```

Example

```
#include <stdio.h>
int main() {
int N;
printf("Enter a number to generate its multiplication table: ");
scanf("%d", &N);
int i = 1;
do {
printf("%d x %d = %d\n", N, i, N * i);
i++;
} while (i<= 10);
return 0;  }
```

Output:

```
Please enter a number to generate its multiplication table: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
```

7 x 8 = 56

7 x 9 = 63

7 x 10 = 70

while loop

While loop is also known as a pre-tested loop. In general, a while loop allows a part of the code to be executed multiple times depending upon a given boolean condition. It can be viewed as a repeating if statement. The while loop is mostly used in the case where the number of iterations is not known in advance.

Syntax of while loop in C language

```
while(condition){
//code to be executed
}
```

Example of the while loop in C language

```
#include<stdio.h>
int main(){
int i=1,number=0,b=9;
printf("Enter a number: ");
scanf("%d",&number);
while(i<=10){
printf("%d \n",(number*i));
i++;
}
return 0;
}
```

Output

Enter a number: 50

50

100

150

200

250

300

350

400

450

500

for loop

The for loop in C language is used to iterate the statements or a part of the program several times. It is frequently used to traverse the data structures like the array and linked list.

. Syntax of for loop in C

The syntax of for loop in c language is given below:

```
for(Expression 1; Expression 2; Expression 3){
//code to be executed
}
```

Example

```
#include<stdio.h>
int main(){
int i=1,number=0;
```

```
        printf("Enter a number: ");
    scanf("%d",&number);
        for(i=1;i<=10;i++){
        printf("%d \n",(number*i));
        }
        return 0;
        }
```

Output

```
Enter a number: 2
2
4
6
8
10
12
14
16
18
20
```

break statement

The break is a keyword in C which is used to bring the program control out of the loop. The break statement is used inside loops or switch statement. The break statement breaks the loop one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops.

Syntax:

```
//loop or switch case
break;
```

Example:

```
#include <stdio.h>
int main() {
int i = 1;
while (i<= 10) {
if (i == 5) {
break; // Exit the loop when i becomes 5
}
printf("%d ", i);
i++;
}
printf("\n");
return 0;
}
```

Output

1 2 3 4

continue statement

The continue statement in C language is used to bring the program control to the beginning of the loop. The continue statement skips some lines of code inside the loop and continues with the next iteration. It is mainly used for a condition so that we can skip some code for a particular condition.

Syntax:

```
//loop statements
continue;
    //some lines of the code which is to be skipped
```

Continue statement example

```
#include<stdio.h>
int main(){
int i=1;//initializing a local variable
//starting a loop from 1 to 10
for(i=1;i<=10;i++){
if(i==5){//if value of i is equal to 5, it will continue the loop
continue;
}
printf("%d \n",i);
}//end of for loop
return 0;
```

Output

```
1
2
3
4
6
7
8
9
10
```

goto statement

The goto statement is known as jump statement in C. As the name suggests, goto is used to transfer the program control to a predefined label. The goto statment can be used to repeat some part of the code for a particular condition. It can also be used to break the multiple loops which can't be done by using a single break statement.

Syntax:

```
label:
//some part of the code;
goto label;
```

goto example:

```
#include <stdio.h>
int main()
{
  int num,i=1;
  printf("Enter the number whose table you want to print?");
  scanf("%d",&num);
  table:
 printf("%d x %d = %d\n",num,i,num*i);
  i++;
  if(i<=10)
  goto table;
}
```

Output:

Enter the number whose table you want to print?10

10 x 1 = 10

10 x 2 = 20

10 x 3 = 30

10 x 4 = 40

10 x 5 = 50

10 x 6 = 60

10 x 7 = 70

10 x 8 = 80

10 x 9 = 90

10 x 10 = 100

Array

An array is defined as the collection of similar type of data items stored at contiguous memory locations. Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc. It also has the capability to store the collection of derived data types, such as pointers, structure, etc. The array is the simplest data structure where each data element can be randomly accessed by using its index number.

Properties of Array

The array contains the following properties.

- Each element of an array is of same data type and carries the same size, i.e., int = 4 bytes.
- Elements of the array are stored at contiguous memory locations where the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Declaration of Array

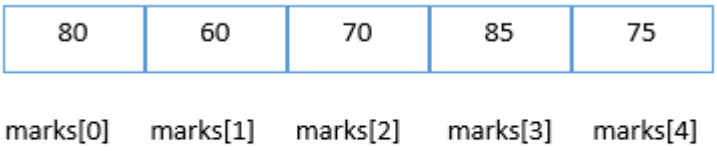
We can declare an array in the c language in the following way.

data_type array_name[array_size];

Initialization of Array

The simplest way to initialize an array is by using the index of each element. We can initialize each element of the array by using the index. Consider the following example.

1. marks[0]=80;//initialization of array
2. marks[1]=60;
3. marks[2]=70;
4. marks[3]=85;
5. marks[4]=75;



Initialization of Array

C array example

```
#include<stdio.h>
int main(){
int i=0;
int marks[5];//declaration of array
marks[0]=80;//initialization of array
marks[1]=60;
marks[2]=70;
marks[3]=85;
marks[4]=75;
//traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
}//end of for loop
    return 0;
}
```

Output

```
80
60
70
85
75
```

C Array: Declaration with Initialization

We can initialize the c array at the time of declaration. Let's see the code.

```
int marks[5]={20,30,40,50,60};
```

Example

```
#include<stdio.h>
int main(){
int i=0;
int marks[5]={20,30,40,50,60};//declaration and initialization of array
 //traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
}
return 0;
}
```

Output

```
20
30
40
50
60
```

Two Dimensional Array

The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database lookalike data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

Declaration of two dimensional Array

The syntax to declare the 2D array is given below.

```
data_type array_name[rows][columns];
```

Consider the following example.

```
int twodimen[4][3];
```

Initialization of 2D Array

In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously. However, this will not work with 2D

arrays. We will have to define at least the second dimension of the array. The two-dimensional array can be declared and defined in the following way.

```
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
```

Two-dimensional array example

```
#include<stdio.h>
int main(){
int i=0,j=0;
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
//traversing 2D array
for(i=0;i<4;i++){
 for(j=0;j<3;j++){
   printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);
 }//end of j
}//end of i
    return 0;
}
```

Output

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6
```

Functions

In c, we can divide a large program into the basic building blocks known as function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the C program. In other words, we can say that the collection of functions creates a program. The function is also known as *procedure*or *subroutine*in other programming languages.

Function Aspects:
There are three aspects of a C function.

- Function declaration A function must be declared globally in a c program to tell the compiler about the function name, function parameters, and return type.
- Function call Function can be called from anywhere in the program. The parameter list must not differ in function calling and function declaration. We must pass the same number of functions as it is declared in the function declaration.
- Function definition It contains the actual statements which are to be executed. It is the most important aspect to which the control comes when the function is called. Here, we must notice that only one value can be returned from the function.

SN	C function aspects	Syntax
1	Function declaration	return_type function_name (argument list);
2	Function call	function_name (argument_list)
3	Function definition	return_type function_name (argument list) {function body;}

The syntax of creating function in c language is given below:

```
return_type function_name(data_type parameter...){
//code to be executed
}
```

Types of Functions
There are two types of functions in C programming:

1. Library Functions: are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc.
2. User-defined functions: are the functions which are created by the C programmer, so that he/she can use it many times. It reduces the complexity of a big program and optimizes the code.

Example:

```
#include <stdio.h>
// Function declaration
void myFunction();

int main() {
  myFunction();  // call the function
  return 0;
}
// Function definition
void myFunction() {
  printf("I just got executed!");
}
```

Output:

I just got executed!

Return Value

A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

Example without return value:

```
void hello(){
printf("hello c");
}
```

If you want to return any value from the function, you need to use any data type such as int, long, char, etc. The return type depends on the value to be returned from the function.

Example with return value:

```
int get(){
    return 10;
}
```

Different aspects of function calling

A function may or may not accept any argument. It may or may not return any value. Based on these facts, There are four different aspects of function calls.

1. function without arguments and without return value
2. function without arguments and with return value
3. function with arguments and without return value
4. function with arguments and with return value

1.Function without argument and return value

```
#include<stdio.h>
void sum();
void main()
    {
   printf("\nGoing to calculate the sum of two numbers:");
   sum();
}
void sum()
{
   int a,b;
   printf("\nEnter two numbers");
   scanf("%d %d",&a,&b);       printf("The sum is %d",a+b);
}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10
24

The sum is 34

2.Function without argument and with return value

```
#include<stdio.h>
int sum();
void main()
{
  printf("Going to calculate the area of the square\n");
  float area = square();
   printf("The area of the square: %f\n",area);
```

```
    }
    int square()
    {
   float side;
  printf("Enter the length of the side in meters: ");
   scanf("%f",&side);
   return side * side;
}
```

Output

```
Going to calculate the area of the square
Enter the length of the side in meters: 10
The area of the square: 100.000000
```

3. Function with argument and without return value

```
#include<stdio.h>
void average(int, int, int, int, int);
void main()
{
   int a,b,c,d,e;
   printf("\nGoing to calculate the average of five numbers:");
   printf("\nEnter five numbers:");
   scanf("%d %d %d %d %d",&a,&b,&c,&d,&e);
   average(a,b,c,d,e);
}
    void average(int a, int b, int c, int d, int e)
{
   float avg;
   avg = (a+b+c+d+e)/5;
       printf("The average of given five numbers : %f",avg);
    }
```

4.Function with argument and with return value

```
#include<stdio.h>
int even_odd(int);
void main()
{
 int n,flag=0;
 printf("\nGoing to check whether a number is even or odd");
 printf("\nEnter the number: ");
 scanf("%d",&n);
```

```
 flag = even_odd(n);
if(flag == 0)
{
   printf("\nThe number is odd");
 }
 else
 {
   printf("\nThe number is even");
 }
}
int even_odd(int n)
{
   if(n%2 == 0)
  {
       return 1;
        }
   else
   {
       return 0;
   }
    }
```

Output:

```
Going to check whether a number is even or odd
Enter the number: 100
The number is even
```

Call by value and Call by reference

There are two methods to pass the data into the function in C language, i.e., call by value and call by reference.

Call by value

- In call by value method, the value of the actual parameters is copied into the formal parameters. In other words, we can say that the value of the variable is used in the function call in the call by value method.
- In call by value method, we can not modify the value of the actual parameter by the formal parameter.
- In call by value, different memory is allocated for actual and formal parameters since the value of the actual parameter is copied into the formal parameter.
- The actual parameter is the argument which is used in the function call whereas formal parameter is the argument which is used in the function definition.

Example:

```
#include<stdio.h>
void change(int num) {
    printf("Before adding value inside function num=%d \n",num);
    num=num+100;        printf("After adding value inside function num=%d \n", num);

     }
int main() {
   int x=100;
  printf("Before function call x=%d \n", x);
   change(x);//passing value in function
    printf("After function call x=%d \n", x);
     return 0;
}
```

Output:
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=100

Call by reference

- In call by reference, the address of the variable is passed into the function call as the actual parameter.
- The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.
- In call by reference, the memory allocation is similar for both formal parameters and actual parameters. All the operations in the function are performed on the value stored at the address of the actual parameters, and the modified value gets stored at the same address.

Example:

```
    #include <stdio.h>
    void swap(int *, int *); //prototype of the function
int main()
    {
        int a = 10;
        int b = 20;
```

```
    printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing t
  he value of a and b in main
    swap(&a,&b);
    printf("After swapping values in main a = %d, b = %d\n",a,b); // The values of
  actual parameters do change in call by reference, a = 10, b = 20
  }
  void swap (int *a, int *b)
  {
    int temp;
    temp = *a;
    *a=*b;
 *b=temp;      printf("After swapping values in function a = %d, b = %d\n",*a,*b); /
/ Formal parameters, a = 20, b = 10
}
```

Output
Before swapping the values in main a = 10, b = 20
After swapping values in function a = 20, b = 10
After swapping values in main a = 20, b = 10

Difference between call by value and call by reference in c

No.	Call by value	Call by reference
1	A copy of the value is passed into the function	An address of value is passed into the function
2	Changes made inside the function is limited to the function only. The values of the actual parameters do not change by changing the formal parameters.	Changes made inside the function validate outside of the function also. The values of the actual parameters do change by changing the formal parameters.
3	Actual and formal arguments are created at the different memory location	Actual and formal arguments are created at the same memory location

Passing Array to Function

As we know that the array_name contains the address of the first element. Here, we must notice that we need to pass only the name of the array in the function which is intended to accept an array. The array defined as the formal parameter will

automatically refer to the array specified by the array name defined as an actual parameter.

Consider the following syntax to pass an array to the function.
functionname(arrayname);//passing array

Methods to declare a function that receives an array as an argument

There are 3 ways to declare the function which is intended to receive an array as an argument.

First way:
1. return_type function(type arrayname[])

Second way:
1. return_type function(type arrayname[SIZE])

Third way:
1. return_type function(type *arrayname)

Example

```
#include<stdio.h>
int minarray(int arr[],int size){
int min=arr[0];
int i=0;
for(i=1;i<size;i++){
if(min>arr[i]){
min=arr[i];
}
    }//end of for
    return min;
    }//end of function
int main(){
int i=0,min=0;
    int numbers[]={4,5,7,3,8,9};//declaration of array
      min=minarray(numbers,6);//passing array with size
    printf("minimum number is %d \n",min);
    return 0;
    }
```

Output
minimum number is 3

Recursion

Recursion is the process which comes into existence when a function calls a copy of itself to work on a smaller problem. Any function which calls itself is called recursive function, and such function calls are called recursive calls. Recursion involves several numbers of recursive calls. However, it is important to impose a termination condition of recursion.

Example:

```
#include <stdio.h>
int fact (int);
int main()
{
    int n,f;
    printf("Enter the number whose factorial you want to calculate?");
    scanf("%d",&n);
    f = fact(n);
   printf("factorial = %d",f);
     }
     int fact(int n)
     {
         if (n==0)
         {
             return 0;
         }
         else if ( n == 1)
    {
        return 1;
         }
         else
    {
             return n*fact(n-1);
         }
     }
```

Output

```
Enter the number whose factorial you want to calculate?5
factorial = 120
```

Strings

The string can be defined as the one-dimensional array of characters terminated by a null ('\0'). The character array or the string is used to manipulate text such as word or sentences. Each character in the array occupies one byte of memory, and the last character must always be 0. The termination character ('\0') is important in a string since it is the only way to identify where the string ends. When we define a string as char s[10], the character s[10] is implicitly initialized with the null in the memory.
There are two ways to declare a string in c language.

1. By char array
2. By string literal

Example of declaring string by char array in C language.
char ch[10]={'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0'};

Define the string by the string literal in C language. For example:

char ch[]="javatpoint";

Difference between char array and string literal

There are two main differences between char array and literal.

1. We need to add the null character '\0' at the end of the array by ourself whereas, it is appended internally by the compiler in the case of the character array.
2. The string literal cannot be reassigned to another set of characters whereas, we can reassign the characters of the array.

String Example in C

```
#include <stdio.h>
int main() {
  char message[] = "Good to see you,";
  char fname[] = "John";
  printf("%s %s!", message, fname);
  return 0;
}
```

Output:
Good to see you, John

String Functions

C also has many useful string functions, which can be used to perform certain operations on strings.

To use them, you must include the <string.h> header file in your program:

String Length

For example, to get the length of a string, you can use the strlen() function:

Example:

```
char alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
printf("%d", strlen(alphabet));
```

output:

26

Concatenate Strings

To concatenate (combine) two strings, you can use the strcat() function:

Example

```
char str1[20] = "Hello ";
char str2[] = "World!";

// Concatenate str2 to str1 (result is stored in str1)
strcat(str1, str2);

// Print str1
printf("%s", str1);
```

output:

Hello World!

Copy Strings

To copy the value of one string to another, you can use the strcpy() function:

Example

```
char str1[20] = "Hello World!";
char str2[20];
```

```
// Copy str1 to str2
strcpy(str2, str1);

// Print str2
printf("%s", str2);
```

output:

Hello World!

Compare Strings

To compare two strings, you can use the strcmp() function.

It returns 0 if the two strings are equal, otherwise a value that is not 0:

Example

```
char str1[] = "Hello";
char str2[] = "Hello";
char str3[] = "Hi";

// Compare str1 and str2, and print the result
printf("%d\n", strcmp(str1, str2));  // Returns 0 (the strings are equal)

// Compare str1 and str3, and print the result
printf("%d\n", strcmp(str1, str3));  // Returns -4 (the strings are not equal)
```

output:

0

-4

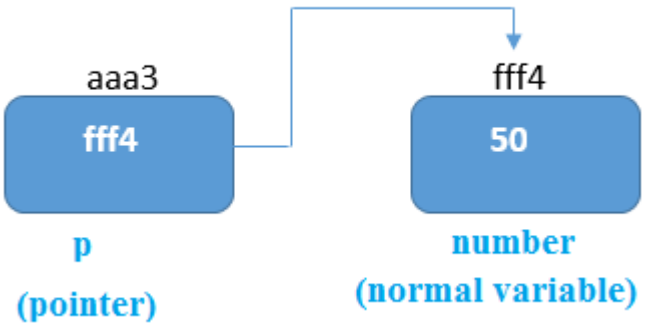
Pointers

The pointer in C language is a variable which stores the address of another variable. This variable can be of type int, char, array, function, or any other pointer. The size of the pointer depends on the architecture. However, in 32-bit architecture the size of a pointer is 2 byte.

Declaring a pointer
The pointer in c language can be declared using * (asterisk symbol). It is also known as indirection pointer used to dereference a pointer.

int *a;//pointer to int
char *c;//pointer to char
int n = 10;
int* p = &n; // Variable p of type pointer is pointing to the address of the variable n of type integer.

Pointer Example



As in the above figure, pointer variable stores the address of number variable, i.e., fff4. The value of number variable is 50. But the address of pointer variable p is aaa3.
By the help of * (indirection operator), we can print the value of pointer variable p.

```
#include<stdio.h>
int main(){
    int number=50;
    int *p;
    p=&number;//stores the address of number variable
    printf("Address of p variable is %x \n",p); // p contains the address of the number t
    herefore printing p gives the address of number.
    printf("Value of p variable is %d \n",*p); // As we know that * is used to dereferen
    ce a pointer therefore if we print *p, we will get the value stored at the address con
    tained by p.
    return 0;
    }
```

Output:

```
Address of number variable is fff4
Address of p variable is fff4
Value of p variable is 50
```

Double Pointer (Pointer to Pointer)

As we know that, a pointer is used to store the address of a variable in C. Pointer reduces the access time of a variable. However, In C, we can also define a pointer to store the address of another pointer. Such pointer is known as a double pointer (pointer to pointer). The first pointer is used to store the address of a variable whereas the second pointer is used to store the address of the first pointer.

The syntax of declaring a double pointer is given below.

```
int **p; // pointer to a pointer which is pointing to an integer.
```

Example:

```
#include<stdio.h>
int main(){
    int number=50;
    int *p;//pointer to int
    int **p2;//pointer to pointer
    p=&number;//stores the address of number variable
    p2=&p;
    printf("Address of number variable is %x \n",&number);
    printf("Address of p variable is %x \n",p);
    printf("Value of *p variable is %d \n",*p);
printf("Address of p2 variable is %x \n",p2);
    printf("Value of **p2 variable is %d \n",*p);
    return 0;
    }
```

Output

```
Address of number variable is fff4
Address of p variable is fff4
Value of *p variable is 50
Address of p2 variable is fff2
Value of **p variable is 50
```

Pointer Arithmetic

We can perform arithmetic operations on the pointers like addition, subtraction, etc. However, as we know that pointer contains the address, the result of an arithmetic operation performed on the pointer will also be a pointer if the other operand is of type integer. In pointer-from-pointer subtraction, the result will be an integer value. Following arithmetic operations are possible on the pointer in C language:

1. Increment
2. Decrement
3. Addition
4. Subtraction

1.Incrementing Pointer

If we increment a pointer by 1, the pointer will start pointing to the immediate next location. This is somewhat different from the general arithmetic since the value of the pointer will get increased by the size of the data type to which the pointer is pointing.

```
#include<stdio.h>
int main(){
int number=50;
    int *p;//pointer to int
     p=&number;//stores the address of number variable
    printf("Address of p variable is %u \n",p);
    p=p+1;
    printf("After increment: Address of p variable is %u \n",p); // in our case, p will ge
    t incremented by 4 bytes.
    return 0;
```

Output
Address of p variable is 3214864300
After increment: Address of p variable is 3214864304

2.Decrementing Pointer in C

Like increment, we can decrement a pointer variable. If we decrement a pointer, it will start pointing to the previous location.

```
    #include <stdio.h>
    void main(){
    int number=50;
    int *p;//pointer to int
    p=&number;//stores the address of number variable
    printf("Address of p variable is %u \n",p);
    p=p-1;
    printf("After decrement: Address of p variable is %u \n",p); // P will now point to
    the immidiate previous location.
}
```

Output
Address of p variable is 3214864300
After decrement: Address of p variable is 3214864296

3.Pointer Addition

We can add a value to the pointer variable

```
#include<stdio.h>
    int main(){
int number=50;
int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p+3;   //adding 3 to pointer variable
printf("After adding 3: Address of p variable is %u \n",p);       return 0;
}
```

Output
Address of p variable is 3214864300
After adding 3: Address of p variable is 3214864312

.

4.Pointer Subtraction

Like pointer addition, we can subtract a value from the pointer variable. Subtracting any number from a pointer will give an address.

```
    #include<stdio.h>
int main(){
    int number=50;
    int *p;//pointer to int
    p=&number;//stores the address of number variable
    printf("Address of p variable is %u \n",p);
    p=p-3; //subtracting 3 from pointer variable
    printf("After subtracting 3: Address of p variable is %u \n",p);
    return 0;
    }
```

Output
Address of p variable is 3214864300
After subtracting 3: Address of p variable is 3214864288

Pointer to Array

We can also use pointers to access arrays.

Example

```
#include <stdio.h>

int main() {
  int myNumbers[4] = {25, 50, 75, 100};
  int i;

  for (i = 0; i < 4; i++) {
    printf("%d\n", myNumbers[i]);
  }

  return 0;
}
```

Output:
25
50
75
100

Pointer to Function

Function pointer is a special pointer that points to a function. Yes a pointer can point to any object in C. Instead pointing at variable, a function pointer points at executable code.

Example:

```
#include <stdio.h>


/* Function declarations */

int add(int, int);

int sub(int, int);


int main()

{

   int sum, diff;
```

```
    int (* arith)(int, int);    // Function pointer declaration
    arith = add;                // Function pointer arith points to add()
    sum   = arith(10, 20);      // Call add() using function pointer
    arith = sub;                // Function pointer arith points to sub()
    diff  = arith(100, 30);     // Call sub() using function pointer


    printf("10+20  => %d\n", sum);
    printf("100-30 => %d\n", diff);
    return 0;
}
/* Function definition */
int add(int num1, int num2)
{
    return (num1 + num2);
}
int sub(int num1, int num2)
{
    return (num1 - num2);
}
```

Output –

```
10+20  => 30
100-30 => 70
```

Types of pointer

Flowing are the types of pointers:

1. void pointer

A pointer to void means a generic pointer that can point to any data type. We can assign the address of any data type to the void pointer, and a void pointer can be assigned to any type of the pointer without performing any explicit typecasting.

Syntax of void pointer

```
void *pointer name;
```

Example:

Output
10

```
#include <stdio.h>

int main()

{

    int a = 10;

    void* ptr = &a;

    printf("%d", *(int*)ptr);

    return 0;

}
```

2.NULL Pointer

The Null Pointer is the pointer that does not point to any location but NULL.

Syntax of Null Pointer Declaration in C

```
type pointer_name = NULL;
type pointer_name = 0;
```

Example:

```
#include <stdio.h>

int main()
{
    // declaring null pointer
    int* ptr = NULL;

    // derefencing only if the pointer have any value
    if (ptr == NULL) {
        printf("Pointer does not point to anything");
    }
    else {
        printf("Value pointed by pointer: %d", *ptr);
    }
```

```
    return 0;
}
```

Output
Pointer does not point to anything

3.Dangling Pointer

A pointer pointing to a memory location that has been deleted (or freed) is called a dangling pointer. Such a situation can lead to unexpected behavior in the program and also serve as a source of bugs in C programs.

```
#include <stdio.h>

#include <stdlib.h>

int main()

{

    int* ptr = (int*)malloc(sizeof(int));

    // After below free call, ptr becomes a dangling pointer

    free(ptr);

    printf("Memory freed\n");

    // removing Dangling Pointer

    ptr = NULL;

    return 0;

}
```

Output
Memory freed

4.Wild pointer

A pointer that has not been initialized to anything (not even NULL) is known as a wild pointer. The pointer may be initialized to a non-NULL garbage value that may not be a valid address.

Syntax
dataType *pointerName;

Example:

```
#include <stdio.h>

int main()

{

   int* p; /* wild pointer */

   int x = 10;

   // Accessing the value pointed by 'p'

   printf("Value pointed by 'p' is: %d\n", *p);

   return 0;

}
```

Output
Segmentation Fault (SIGSEGV)
timeout: the monitored command dumped core
/bin/bash: line 1:   32 Segmentation fault

Structure

Structure in c is a user-defined data type that enables us to store the collection of different data types. Each element of a structure is called a member. Structures ca; simulate the use of classes and templates as it can store various information. The ,struct keyword is used to define the structure.

Syntax:

```
    struct structure_name
{
```

```
    data_type member1;
    data_type member2;
        .
        .
        data_type memeberN;
    };
```

Example

```
#include<stdio.h>
#include <string.h>
    struct employee
    {   int id;
        char name[50];
    }e1;  //declaring e1 variable for structure
    int main( )
    {
      //store first employee information
      e1.id=101;
      strcpy(e1.name, "Sonoo Jaiswal");//copying string into char array
      //printing first employee information
      printf( "employee 1 id : %d\n", e1.id);
      printf( "employee 1 name : %s\n", e1.name);
    return 0;
    }
```

Output:
employee 1 id : 101
employee 1 name : Sonoo Jaiswal

Array of Structures

An array of structres in C can be defined as the collection of multiple structures variables where each variable contains information about different entities. The array of structures in C are used to store information about multiple entities of different data types. The array of structures is also known as the collection of structures.

Example:

```
#include<stdio.h>
    #include <string.h>
struct student{
```

```
int rollno;
    char name[10];
};
    int main(){
    int i;
    struct student st[5];
    printf("Enter Records of 5 students");
    for(i=0;i<5;i++){
    printf("\nEnter Rollno:");
    scanf("%d",&st[i].rollno);
    printf("\nEnter Name:");
    scanf("%s",&st[i].name);
    }
    printf("\nStudent Information List:");
    for(i=0;i<5;i++){
    printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
}
      return 0;
    }
```

Output:

```
Enter Records of 5 students
Enter Rollno:1
Enter Name:Sonoo
Enter Rollno:2
Enter Name:Ratan
Enter Rollno:3
Enter Name:Vimal
Enter Rollno:4
Enter Name:James
Enter Rollno:5
Enter Name:Sarfraz

Student Information List:
Rollno:1, Name:Sonoo
Rollno:2, Name:Ratan
Rollno:3, Name:Vimal
Rollno:4, Name:James
Rollno:5, Name:Sarfraz
```

Nested Structure

C provides us the feature of nesting one structure within another structure by using which, complex data types are created. Consider the following program.

```
#include<stdio.h>
struct address
{
   char city[20];
   int pin;
   char phone[14];
};
struct employee
{
   char name[20];
   struct address add;
};
void main ()
{
   struct employee emp;
   printf("Enter employee information?\n");
  scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add.pin, emp.add.phone);
   printf("Printing the employee information....\n");
printf("name: %s\nCity: %s\nPincode: %d\nPhone: %s",emp.name,emp.add.city,e
mp.add.pin,emp.add.phone);
    }
```

Output
Enter employee information?

Arun

Delhi

110001

1234567890

Printing the employee information....

name: Arun

City: Delhi

Pincode: 110001

Phone: 1234567890

Structure Pointer

The structure pointer points to the address of a memory block where the Structure is being stored. Like a pointer that tells the address of another variable of any data type (int, char, float) in memory.

Declare a Structure Pointer

```
struct structure_name *ptr;
```

Initialization of the Structure Pointer

```
ptr = &structure_variable;
```

Access Structure member using pointer:

There are two ways to access the member of the structure using Structure pointer:

1. Using (*) asterisk or indirection operator and dot (.) operator.
2. Using arrow (->) operator or membership operator.

Program to access the structure member using structure pointer and the dot operator

```
#include <stdio.h>

    // create a structure Subject using the struct keyword
struct Subject
{
   // declare the member of the Course structure
   char sub_name[30];
   int sub_id;
   char sub_duration[50];
   char sub_type[50];
};
int main()
{
   struct Subject sub; // declare the Subject variable
   struct Subject *ptr; // create a pointer variable (*ptr)
       ptr = ⊂ /* ptr variable pointing to the address of the structure variable sub *
   strcpy (sub.sub_name, " Computer Science");
       sub.sub_id = 1201;
   strcpy (sub.sub_duration, "6 Months");
strcpy (sub.sub_type, " Multiple Choice Question");
   // print the details of the Subject;
   printf (" Subject Name: %s\t ", (*ptr).sub_name);
         printf (" \n Subject Id: %d\t ", (*ptr).sub_id);
          printf (" \n Duration of the Subject: %s\t ", (*ptr).sub_duration);
        printf (" \n Type of the Subject: %s\t ", (*ptr).sub_type);
   return 0;
     }
```

Output:
Subject Name: Computer Science
Subject Id: 1201
Duration of the Subject: 6 Months
Type of the Subject: Multiple Choice Question

Program to access the structure member using structure pointer and arrow (->) operator

```
#include <stdio.h>

// create Employee structure
struct Employee
{
   // define the member of the structure
   char name[30];
   int id;
   int age;
   char gender[30];
   char city[40];
};

// define the variables of the Structure with pointers
struct Employee emp1, emp2, *ptr1, *ptr2;

int main()
{
      // store the address of the emp1 and emp2 structure variable
   ptr1 = &emp1;
   ptr2 = &emp2;
   printf (" Enter the name of the Employee (emp1): ");
   scanf (" %s", &ptr1->name);
printf (" Enter the id of the Employee (emp1): ");
   scanf (" %d", &ptr1->id);
   printf (" Enter the age of the Employee (emp1): ");
   scanf (" %d", &ptr1->age);
   printf (" Enter the gender of the Employee (emp1): ");
   scanf (" %s", &ptr1->gender);
   printf (" Enter the city of the Employee (emp1): ");
   scanf (" %s", &ptr1->city);
      printf (" \n Second Employee: \n");
     printf (" Enter the name of the Employee (emp2): ");
      scanf (" %s", &ptr2->name);

   printf (" Enter the id of the Employee (emp2): ");
   scanf (" %d", &ptr2->id);
   printf (" Enter the age of the Employee (emp2): ");
scanf (" %d", &ptr2->age);
```

```
        printf (" Enter the gender of the Employee (emp2): ");
            scanf (" %s", &ptr2->gender);
            printf (" Enter the city of the Employee (emp2): ");
        scanf (" %s", &ptr2->city);

            printf ("\n Display the Details of the Employee using Structure Pointer");
            printf ("\n Details of the Employee (emp1) \n");
        printf(" Name: %s\n", ptr1->name);
        printf(" Id: %d\n", ptr1->id);
        printf(" Age: %d\n", ptr1->age);
        printf(" Gender: %s\n", ptr1->gender);
        printf(" City: %s\n", ptr1->city);

        printf ("\n Details of the Employee (emp2) \n");
        printf(" Name: %s\n", ptr2->name);
        printf(" Id: %d\n", ptr2->id);
        printf(" Age: %d\n", ptr2->age);
        printf(" Gender: %s\n", ptr2->gender);
    printf(" City: %s\n", ptr2->city);
    return 0;
     }
```

Output:

```
Enter the name of the Employee (emp1): John
 Enter the id of the Employee (emp1): 1099
 Enter the age of the Employee (emp1): 28
 Enter the gender of the Employee (emp1): Male
 Enter the city of the Employee (emp1): California

 Second Employee:
 Enter the name of the Employee (emp2): Maria
 Enter the id of the Employee (emp2): 1109
 Enter the age of the Employee (emp2): 23
 Enter the gender of the Employee (emp2): Female
 Enter the city of the Employee (emp2): Los Angeles

 Display the Details of the Employee using Structure Pointer
 Details of the Employee (emp1)
 Name: John
 Id: 1099
 Age: 28
 Gender: Male
 City: California
 Details of the Employee (emp2) Name: Maria
 Id: 1109
 Age: 23
Gender: Female
 City: Los Angele
```

Union

Union can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location. The union can also be defined as many members, but only one member can contain a value at a particular point in time.
Union is a user-defined data type, but unlike structures, they share the same memory location.

Example:

```
#include <stdio.h>
    struct store
{
   double price;
   union
   {
      struct{
      char *title;
          char *author;
          int number_pages;
      } book;

      struct {
      int color;
      int size;
      char *design;
          } shirt;
   }item;
};
  int main()
{
   struct store s;
        s.item.book.title = "C programming";
        s.item.book.author = "John";
        s.item.book.number_pages = 189;
        printf("Size is %ld", sizeof(s));
   return 0;
}
```

typedef

The typedef is a keyword used in C programming to provide some meaningful names to the already existing variable in the C program. It behaves similarly as we define the alias for the commands. In short, we can say that this keyword is used to redefine the name of an already existing variable.

Syntax of typedef

```
typedef <existing_name> <alias_name>
```

Example.

```
#include <stdio.h>
int main()
{
    typedef unsigned int unit;
    unit i,j;
    i=10;
    j=20;
    printf("Value of i is :%d",i);
    printf("\nValue of j is :%d",j);
    return 0;
    }
```

Output
Value of i is :10
Value of j is :20

Dynamic memory allocation

The concept of dynamic memory allocation in c language *enables the C programmer to allocate memory at runtime*. Dynamic memory allocation in c language is possible by 4 functions of stdlib.h header file.

1. malloc()
2. calloc()
3. realloc()
4. free()

difference between static memory allocation and dynamic memory allocation.

static memory allocation	**dynamic memory allocation**
memory is allocated at compile time.	memory is allocated at run time.
memory can't be increased while executing program.	memory can be increased while execut
used in array.	used in linked list.

1. **malloc()**

The malloc() function allocates single block of requested memory. It doesn't initialize memory at execution time, so it has garbage value initially.
It returns NULL if memory is not sufficient.

The syntax of malloc() function is given below:

```
ptr=(cast-type*)malloc(byte-size)
```

example of malloc() function.

```
#include<stdio.h>
#include<stdlib.h>
int main(){
   int n,i,*ptr,sum=0;
  printf("Enter number of elements: ");
  scanf("%d",&n);
  ptr=(int*)malloc(n*sizeof(int));  //memory allocated using malloc
  if(ptr==NULL)
  {
     printf("Sorry! unable to allocate memory");
     exit(0);
  }
  printf("Enter elements of array: ");
  for(i=0;i<n;++i)
  {
     scanf("%d",ptr+i);
     sum+=*(ptr+i);
      }
  printf("Sum=%d",sum);
  free(ptr);
return 0;
}
```

Output
Enter elements of array: 3
Enter elements of array: 10
10
10
Sum=30

2. calloc()

The calloc() function allocates multiple block of requested memory.It initially initialize all bytes to zero.It returns NULL if memory is not sufficient.

The syntax of calloc() function is given below:

```
ptr=(cast-type*)calloc(number, byte-size)
```

example of calloc() function.

```
#include<stdio.h>
#include<stdlib.h>
int main(){
 int n,i,*ptr,sum=0;
printf("Enter number of elements: ");
  scanf("%d",&n);
```

```
        ptr=(int*)calloc(n,sizeof(int));  //memory allocated using calloc
      if(ptr==NULL)
      {
            printf("Sorry! unable to allocate memory");
         exit(0);
      }
      printf("Enter elements of array: ");
      for(i=0;i<n;++i)
          {
         scanf("%d",ptr+i);
         sum+=*(ptr+i);
      }
      printf("Sum=%d",sum);
      free(ptr);
   return 0;
   }
```

Output
Enter elements of array: 3
Enter elements of array: 10
10
10
Sum=30

3. **realloc()**

If memory is not sufficient for malloc() or calloc(), you can reallocate the memory by realloc() function. In short, it changes the memory size.

Let's see the syntax of realloc() function.

```
ptr=realloc(ptr, new-size)
```

Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int *ptr = (int*) malloc(3 * sizeof(int));
ptr[0] = 1;
ptr[1] = 2;
ptr[2] = 3;

   // resize the memory block to hold 5 integers
ptr = (int*) realloc(ptr, 5 * sizeof(int));
ptr[3] = 4;
ptr[4] = 5;
```

```
        for (int i = 0; i< 5; i++) {
printf("%d ", ptr[i]);
    }
    // free the memory block
    free(ptr);
    return 0;
}
```

Output
1 2 3 4 5

4. **free() function**

The memory occupied by malloc() or calloc() functions must be released by calling free() function. Otherwise, it will consume memory until program exit.

Let's see the syntax of free() function.

```
free(ptr)
```

Storage Classes

Storage classes in C are used to determine the lifetime, visibility, memory location, and initial value of a variable. There are four types of storage classes in C.

1. Automatic
2. External
3. Static
4. Register

1. Automatic

- Automatic variables are allocated memory automatically at runtime.
- The visibility of the automatic variables is limited to the block in which they are defined.
- The scope of the automatic variables is limited to the block in which they are defined.
- The automatic variables are initialized to garbage by default.
- The memory assigned to automatic variables gets freed upon exiting from the block.
- The keyword used for defining automatic variables is auto.
- Every local variable is automatic in C by default.

Example

```
#include <stdio.h>
int main()
{
int a = 10,i;
printf("%d ",++a);
{
int a = 20;
for (i=0;i<3;i++)
{
printf("%d ",a); // 20 will be printed 3 times since it is the local value of a
}
}
printf("%d ",a); // 11 will be printed since the scope of a = 20 is ended.
}
```

Output:

11 20 20 20 11

2. Static

- The variables defined as static specifier can hold their value between the multiple function calls.
- Static local variables are visible only to the function or the block in which they are defined.
- A same static variable can be declared many times but can be assigned at only one time.
- Default initial value of the static integral variable is 0 otherwise null.
- The visibility of the static global variable is limited to the file in which it has declared.
- The keyword used to define static variable is static.

```
#include<stdio.h>
void sum()
{
static int a = 10;
static int b = 24;
printf("%d %d \n",a,b);
a++;
b++;
}
void main()
{
int i;
for(i = 0; i< 3; i++)
{
```

```
    sum(); // The static variables holds their value between multiple function calls.

    }
}
```

Output:
10 24
11 25
12 26

3. **Register**

- The variables defined as the register is allocated the memory into the CPU registers depending upon the size of the memory remaining in the CPU.
- We can not dereference the register variables, i.e., we can not use &operator for the register variable.
- The access time of the register variables is faster than the automatic variables.
- The initial default value of the register local variables is 0.
- The register keyword is used for the variable which should be stored in the CPU register. However, it is compiler?s choice whether or not; the variables can be stored in the register.
- We can store pointers into the register, i.e., a register can store the address of a variable.
- Static variables can not be stored into the register since we can not use more than one storage specifier for the same variable.

Example

```
#include <stdio.h>
int main()
{
register int a; // variable a is allocated memory in the CPU register. The initiald
efault value of a is 0.
printf("%d",a);

}
```

Output:
0

4. **External**

- The external storage class is used to tell the compiler that the variable defined as extern is declared with an external linkage elsewhere in the program.
- The variables declared as extern are not allocated any memory. It is only declaration and intended to specify that the variable is declared elsewhere in the program.
- The default initial value of external integral type is 0 otherwise null.

- We can only initialize the extern variable globally, i.e., we can not initialize the external variable within any block or method.
- An external variable can be declared many times but can be initialized at only once.
- If a variable is declared as external then the compiler searches for that variable to be initialized somewhere in the program which may be extern or static. If it is not, then the compiler will show an error.

Example

```
#include <stdio.h>
int main()
{
extern int a; // Compiler will search here for a variable a defined and initialized so mewhere in the pogram or not.
printf("%d",a);
}
int a = 20;
```

Output
20

Preprocessor Directives

The C preprocessor is a micro processor that is used by compiler to transform your code before compilation. It is called micro preprocessor because it allows us to add macros. All preprocessor directives starts with hash # symbol.

1. **#include**

The #include preprocessor directive is used to paste code of given file into current file. It is used include system-defined and user-defined header files. If included file is not found, compiler renders error.
By the use of #include directive, we provide information to the preprocessor where to look for the header files. There are two variants to use #include directive.

1. #include <filename>
2. #include "filename"

   The #include <filename> tells the compiler to look for the directory where system header files are held.

   The #include "filename" tells the compiler to look in the current directory from where program is running.

#include directive example

Let's see a simple example of #include directive. In this program, we are including stdio.h file because printf() function is defined in this file.

```
#include<stdio.h>
```

```
int main(){
    printf("Hello C");
    return 0;
}
```

Output:

Hello C

2. #define

The C programming language's #define preprocessor directive provides a strong and flexible tool for declaring constants and producing macros. It conducts textual replacement before actual compilation during the pre-processing phase of C programs, where it plays a significant role. By using this functionality, developers may improve their code's readability, maintainability, and effectiveness.

Syntax:

```
#define token value
```

Let's see an example of #define.

```
#include <stdio.h>
#define PI 3.14
main() {
    printf("%f",PI);
}
```

Output:

3.140000

3. #undef

The #undef preprocessor directive is used to undefine the constant or macro defined by #define.

Syntax:

```
#undef token
```

Example:

```
#include <stdio.h>
#define number 15
int square=number*number;
#undef number
main() {
printf("%d",square);
}
```

Output:

225

4. #ifdef

The #ifdef preprocessor directive checks if macro is defined by #define. If yes, it executes the code otherwise #else code is executed, if present.

Syntax:

```
#ifdef MACRO
//code
#endif
```

Example

```
#include <stdio.h>
#include <conio.h>
#define NOINPUT
void main() {
int a=0;
#ifdef NOINPUT
a=2;
#else
```

```
printf("Enter a:");
scanf("%d", &a);
#endif
printf("Value of a: %d\n", a);
getch();
}
```

Output:

Value of a: 2

5. #ifndef

The #ifndef preprocessor directive checks if macro is not defined by #define. If yes, it executes the code otherwise #else code is executed, if present.

Syntax:

```
#ifndef MACRO
//code
#endif
```

Example

```
#include <stdio.h>
#include <conio.h>
#define INPUT
void main() {
int a=0;
#ifndef INPUT
a=2;
#else
printf("Enter a:");
scanf("%d", &a);
#endif
printf("Value of a: %d\n", a);
getch();
}
```

Output:

Enter a:5

Value of a: 5

6.#if

The #if preprocessor directive evaluates the expression or condition. If condition is true, it executes the code otherwise #elseif or #else or #endif code is executed.

Syntax:

```
#if expression
//code
#endif
```

Example

```
#include <stdio.h>
#include <conio.h>
#define NUMBER 0
void main() {
#if (NUMBER==0)
printf("Value of Number is: %d",NUMBER);
#endif
getch();
}
```

Output:

Value of Number is: 0

8.#else

The #else preprocessor directive evaluates the expression or condition if condition of #if is false. It can be used with #if, #elif, #ifdef and #ifndef directives.

Syntax:

```
#if expression
```

```
//if code
#else
    //else code
#endif
```

Example

```
    #include <stdio.h>
    #include <conio.h>
    #define NUMBER 1
    void main() {
    #if NUMBER==0
printf("Value of Number is: %d",NUMBER);
    #else
    print("Value of Number is non-zero");
#endif
    getch();
    }
```

Output:

Value of Number is non-zero

9.#error

The #error preprocessor directive indicates error. The compiler gives fatal error if #error directive is found and skips further compilation process.

Example

Let's see a simple example to use #error preprocessor directive.

```
    #include<stdio.h>
    #ifndef __MATH_H
#error First include then compile
    #else
    void main(){
       float a;
```

```
    a=sqrt(7);
     printf("%f",a);
}
  #endif
```

Output:

Compile Time Error: First include then compile

File Handling

File handling in C enables us to create, update, read, and delete the files stored on the local file system through our C program. The following operations can be performed on a file.

- Creation of the new file
- Opening an existing file
- Reading from the file
- Writing to the file
- Deleting the file

Opening File: fopen()

We must open a file before it can be read, write, or update. The fopen() function is used to open a file.

The syntax of the fopen() is given below.

```
FILE *fopen( const char * filename, const char * mode );
```

We can use one of the following modes in the fopen() function.

Mode	Description
R	opens a text file in read mode
W	opens a text file in write mode
A	opens a text file in append mode
r+	opens a text file in read and write mode
w+	opens a text file in read and write mode
a+	opens a text file in read and write mode

Consider the following example which opens a file in write mode.

```
#include<stdio.h>
void main( )
{
FILE *fp ;
char ch ;
fp = fopen("file_handle.c","r") ;
while ( 1 )
{
ch = fgetc ( fp ) ;
if ( ch == EOF )
break ;
printf("%c",ch) ;
}
fclose (fp ) ;
}
```

Closing File: fclose()

The fclose() function is used to close a file. The file must be closed after performing all the operations on it.

The syntax of fclose() function is given below:

```
int fclose( FILE *fp );
```

Writing File : fputc() function

The fputc() function is used to write a single character into file. It outputs a character to a stream.

Syntax:

```
int fputc(int c, FILE *stream)
```

Example:

```
#include <stdio.h>
main(){
  FILE *fp;
  fp = fopen("file1.txt", "w");//opening file
  fputc('a',fp);//writing single character into file
  fclose(fp);//closing file
}
```

Reading File : fgetc() function

The fgetc() function returns a single character from the file. It gets a character from the stream. It returns EOF at the end of file.

Syntax:

```
int fgetc(FILE *stream)
```

Example:

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
char c;
fp=fopen("myfile.txt","r");
```

```
while((c=fgetc(fp))!=EOF){
printf("%c",c);
}
fclose(fp);
getch();
}
```

Writing File : fputs() function

The fputs() function writes a line of characters into file. It outputs string to a stream.

Syntax:

```
int fputs(const char *s, FILE *stream)
```

Example**:**

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
clrscr();
fp=fopen("myfile2.txt","w");
fputs("hello c programming",fp);
fclose(fp);
getch();
}
```

Reading File : fgets() function

The fgets() function reads a line of characters from file. It gets string from a stream.

Syntax:

```
char* fgets(char *s, int n, FILE *stream)
```

Example:

```
#include<stdio.h>
    #include<conio.h>
    void main(){
    FILE *fp;
char text[300];
clrscr();
fp=fopen("myfile2.txt","r");
printf("%s",fgets(text,200,fp));
fclose(fp);
getch();
}
```

fseek() function

The fseek() function is used to set the file pointer to the specified offset. It is used to write data into file at desired location.

Syntax:

```
int fseek(FILE *stream, long int offset, int whence)
#include <stdio.h>
void main(){
  FILE *fp;
  fp = fopen("myfile.txt","w+");
  fputs("This is javatpoint", fp);
 fseek( fp, 7, SEEK_SET );
  fputs("sonoo jaiswal", fp);
  fclose(fp);
 }
```

rewind() function

The rewind() function sets the file pointer at the beginning of the stream. It is useful if you have to use stream many times.

Syntax:

```
void rewind(FILE *stream)
```

Example:

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
char c;
clrscr();
fp=fopen("file.txt","r");
while((c=fgetc(fp))!=EOF)
{
printf("%c",c);
}
rewind(fp);//moves the file pointer at beginning of the file
while((c=fgetc(fp))!=EOF){
printf("%c",c);
}
fclose(fp);
getch();
}
```